

[CS230] Structuring Machine Learning Projects: Orthogonalization Strategy

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-5. Deep Learning Fundamentals - *Completed*

Chapter 6. Structuring ML Projects (Current Unit)

- **6.1 Orthogonalization Strategy**
 - * Concept: One Knob for One Function
 - * The 4-Step Chain of Assumptions
 - * Mapping Tools to Problems
 - * Why Early Stopping is problematic?
- 6.2 Evaluation Metric (Single Number) - *Upcoming*
- 6.3 Train/Dev/Test Distributions - *Upcoming*

지난 시간 복습 및 연결

우리는 지금까지 딥러닝 모델을 만들고(Build), 학습시키고(Train), 최적화(Optimize)하는 기술적인 방법론을 배웠습니다. 하지만 여러분이 현업에서 리더가 되어 팀을 이끌게 되면 이런 질문에 봉착합니다. "교수님, 성능이 안 나오는데 데이터를 더 모을까요, 층을 더 쌓을까요, 아니면 하이퍼파라미터를 튜닝할까요?" 이때 "일단 다 해봐"라고 말하면 프로젝트는 망합니다. 자원은 유한하기 때문입니다. 오늘 배울 **직교화(Orthogonalization)**는 복잡한 문제 상황에서 무엇부터 해결해야 할지 알려주는 나침반입니다.

1 Unit Overview

□ 핵심 목표

이 단원은 머신러닝 프로젝트를 효율적으로 이끄는 **전략적 사고방식**을 다룹니다.

- **개념:** 시스템의 변수들을 서로 독립적으로(Orthogonal) 제어하여 원하는 효과를 정밀 타격하는 원리를 배웁니다.
- **진단:** 프로젝트 성공을 위한 **4단계 가정(Chain of Assumptions)**을 확립합니다.
- **도구:** 각 문제(Bias, Variance 등)를 해결하기 위한 '**직교화된 도구**'를 매핑합니다.
- **주의:** 조기 종료(Early Stopping)가 왜 직교화 원칙에 위배되는지 분석합니다.

2 Essential Terminology

용어	의미	비유
Orthogonalization	변수 간 독립성 확보	핸들은 방향만, 페달은 속도만 조절함.
Chain of Assumptions	순차적 해결 단계	1 단계 (Train) → 2 단계 (Dev) → 3 단계 (Test).
Orthogonal Tool	한 가지 문제만 해결하는 도구	Bigger Network (Bias 만 잡음).

3 Core Concepts: 한 번에 하나씩

3.1 1. What is Orthogonalization? (직교화란?)

수학적으로 두 벡터가 직교(90도)하면, 한 벡터를 조절해도 다른 벡터에는 영향을 주지 않습니다. 엔지니어링에서도 마찬가지입니다. "하나의 다이얼은 하나의 기능만 조절해야 한다"는 원칙입니다.

□ 자동차 운전 비유 (직관적 비유)

- **Orthogonal (좋은 설계):**
 - 핸들 → 방향 조절 (속도 영향 X)
 - 페달 → 속도 조절 (방향 영향 X)
 - 운전하기 쉽습니다. 원하는 대로 제어 가능합니다.
- **Non-Orthogonal (나쁜 설계):**
 - 핸들을 꺾을 때마다 브레이크가 걸리고 라디오 볼륨이 커진다면?
 - 운전이 불가능합니다. 튜닝도 마찬가지입니다.

3.2 2. The Chain of Assumptions (4단계 진단)

머신러닝 프로젝트는 다음 4단계를 순서대로 통과해야 합니다. 앞 단계가 해결되지 않으면 뒷 단계는 의미가 없습니다.

1. **Fit Training Set:** 훈련 데이터에서 인간 수준 성능을 낸다. (Bias 문제)
2. **Fit Dev Set:** 훈련된 모델이 검증 데이터에서도 잘한다. (Variance 문제)
3. **Fit Test Set:** 검증 데이터 성능이 테스트 데이터와 비슷하다. (Data Mismatch)
4. **Perform in Real World:** 실제 사용자가 만족한다. (Cost Function 오류)

4 Deep Dive: 도구 매핑 (Tool Mapping)

각 단계에서 문제가 발생했을 때, 다른 단계에 부작용을 주지 않고 해당 문제만 해결하는 직교화된 도구를 써야 합니다.

□ Dr. Ng's Prescription Table (전략 가이드)

문제 (Problem)	직교화된 도구 (Good)	비추천 도구 (Bad)
1. High Bias (Train 성능 낮음)	Bigger Network Adam Optimizer	Early Stopping (Bias/Var 둘 다 건드림)
2. High Variance (Dev 성능 낮음)	More Data Regularization (L2, Dropout)	Early Stopping
3. Test Mismatch	Bigger Dev Set	-
4. Real World Fail	Change Cost Function	-

Early Stopping의 딜레마 (오해 방지 가이드)

Early Stopping은 학습을 중간에 멈춥니다.

- 비용함수 J 최소화 중단 → **Bias** 악화
- 가중치 W 증가 억제 → **Variance** 개선

두 가지 효과가 섞여 있어(Coupled), 문제가 생겼을 때 원인을 파악하기 어렵게 만듭니다. 직교화 관점에서는 "Bias는 네트워크 크기로 잡고, Variance는 정규화로 잡는 것"이 더 명확합니다.

5 Implementation: Automated Strategist

직교화는 코드가 아니라 판단 로직입니다. 이를 자동화된 진단 클래스로 구현해봅시다.

```

1 class MLStrategist:
2     def __init__(self, human_err, train_err, dev_err, test_err):
3         self.human = human_err
4         self.train = train_err
5         self.dev = dev_err
6         self.test = test_err
7         self.threshold = 0.02 # 2% 이상 차이면 문제
8
9     def diagnose(self):
10        print("--- Diagnosis Report ---")
11
12        # 1. Bias Check
13        avoidable_bias = self.train - self.human
14        if avoidable_bias > self.threshold:
15            print("[Problem] High Bias (Underfitting)")
16            print("[Action] Increase Network Size, Train Longer.")
17            return # 앞 단계 해결 전엔 뒤를 보지 않음 (Orthogonal)
18
19        # 2. Variance Check
20        variance = self.dev - self.train
21        if variance > self.threshold:
22            print("[Problem] High Variance (Overfitting)")
23            print("[Action] Get More Data, Add Regularization.")
24            return
25
26        # 3. Mismatch Check
27        mismatch = self.test - self.dev
28        if mismatch > self.threshold:
29            print("[Problem] Overfitting to Dev Set")
30            print("[Action] Collect More Dev/Test Data.")
31            return
32

```

```

33     print("[Result] Good Job! Ready for Deployment.")
34
35 # --- 실행 ---
36 if __name__ == "__main__":
37     # 시나리오: 훈련도 안 된 상태
38     strategist = MLStrategist(0.01, 0.15, 0.16, 0.17)
39     strategist.diagnose()

```

Listing 1: ML Project Diagnosis Logic

6 FAQ & Pitfalls

Q. Cost Function은 언제 바꾸나요?

A. 모델의 수치적 성능(Accuracy)은 좋은데, 실제 앱 사용자들의 불만(Real World)이 많을 때입니다. 예를 들어, 고양이 분류기가 야한 사진을 고양이로 분류했다면 정확도가 높아도 치명적입니다. 이때는 Cost Function에 '야한 사진 패널티'를 추가해야 합니다. (과녁 자체를 수정)

Q. Bias를 잡으려고 Regularization을 쓰면 안 되나요?

A. 비추천합니다. Regularization은 모델을 단순하게 만들어 Variance를 줄이는 도구입니다. 부작용으로 Bias가 약간 높아질 수 있습니다. Bias를 잡을 때는 그냥 더 큰 네트워크(Bigger Network)를 쓰는 것이 부작용 없는(Orthogonal) 해결책입니다.

다음 단계 (Next Step)

이제 우리는 문제 해결의 순서와 전략을 알았습니다. 그런데 모델 A는 정확도가 높는데 속도가 느리고, 모델 B는 정확도는 조금 낮는데 속도가 빠릅니다. 도대체 무엇을 선택해야 할까요?

다음 시간에는 애매모호한 상황을 숫자 하나로 정리하여 의사결정 속도를 높이는 [Single Number Evaluation Metric]에 대해 배우겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Orthogonalization:** 변수들을 독립적으로 제어하여 튜닝을 명확하게 만드는 전략.
2. **Sequence:** Train Fit → Dev Fit → Test Fit 순서로 해결한다.
3. **Bias Tool:** Bigger Network, Adam Optimizer.
4. **Variance Tool:** More Data, Regularization.